

TradingMoE: Routing the Right Experts in Evolving Markets

Chang Zhou
University of Science and Technology
of China
Hefei, China
zhouchang21sy@mail.ustc.edu.cn

Xingtong Yu*
The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China
xytu@se.cuhk.edu.hk

Minbin Huang
The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China
huangminbin@link.cuhk.edu.hk

Zhennan Wu
The University of Tokyo
Tokyo, Japan
swwzn714@gmail.com

Yuan Fang
Singapore Management University
Singapore, Singapore
yfang@smu.edu.sg

Hong Cheng
The Chinese University of Hong Kong
Hong Kong, Hong Kong SAR, China
hcheng@se.cuhk.edu.hk

Xinming Zhang*
University of Science and Technology
of China
Hefei, China
xinming@ustc.edu.cn

Abstract

Large language models (LLMs) have shown strong potential for financial analysis and trading, but direct trading remains challenging because the predictive capabilities required can vary across assets, decision fields, and market conditions. Existing LLM-based trading systems either coordinate human-defined external experts or adopt conventional internal Mixture-of-Experts (MoE) routers that do not directly evaluate how individual experts contribute to trading decisions. Moreover, these routers receive no direct signal indicating when an inactive expert has become more suitable as market conditions change. We find that native router scores poorly reflect how much individual experts improve trading decisions, frequently leaving better alternatives unselected. We further reveal that token-specific expert usefulness exhibits a compact low-dimensional structure. Based on these findings, we propose TradingMoE, a trading-oriented sparse MoE that augments a frozen dense LLM with lightweight residual experts. We introduce a Query-Key router that represents the expertise required by each token under the current market context as a low-dimensional query and matches it with learnable expert keys. We further propose a sparse expert selection update mechanism that samples a few inactive experts during training and estimates whether they should replace the weakest expert in the current Top- k route. This mechanism enables the router to update expert selection as market conditions change while preserving sparse computation. Experiments against 22 baselines on stock and cryptocurrency markets show that TradingMoE

improves cumulative return over the best-performing baselines by 30.89% and 30.7%, respectively, while exceeding the corresponding buy-and-hold benchmarks by 37.81% and 80.17% percentage points. Rolling paper-trading experiments further demonstrate that its advantage persists under forward-only deployment. Code is available at <https://anonymous.4open.science/r/TradingMoE-DC52>.

CCS Concepts

• **Computing methodologies** → **Neural networks**; • **Information systems** → **Data mining**; • **Applied computing** → **Economics**.

Keywords

Large language models, ai4trading, mixture-of-experts

ACM Reference Format:

Chang Zhou, Xingtong Yu, Minbin Huang, Zhennan Wu, Yuan Fang, Hong Cheng, and Xinming Zhang. 2026. TradingMoE: Routing the Right Experts in Evolving Markets. In . ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Artificial intelligence has been widely applied to trading, supporting tasks such as return forecasting, asset ranking, and portfolio optimization [15, 22, 26, 30, 47, 49, 55, 58]. More recently, financial large language models (LLMs) have enabled the joint processing of heterogeneous market information [51, 56]. Existing methods use LLMs either to extract signals and support market analysis [11, 17, 48, 57] or to directly generate trading decisions [52, 53, 59]. However, these approaches typically rely on a single dense model or a fixed agentic workflow to process all market states and decision components. Such uniform computation is poorly suited to heterogeneous and non-stationary financial markets, where the predictive capabilities required can vary substantially across assets, decision fields, and market regimes. This limitation motivates methods that can dynamically allocate specialized modeling capacity according to the current trading context.

*Corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

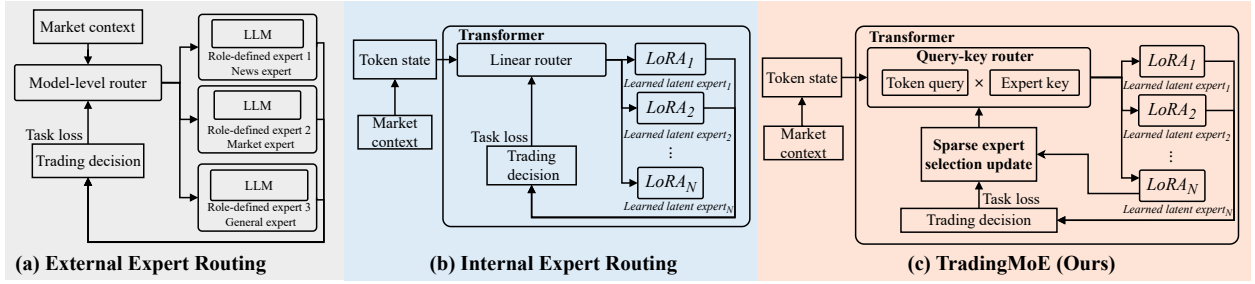


Figure 1: Comparison of expert routing methods.

Existing LLM-based trading systems commonly realize specialization through external expert routing, as shown in Fig. 1(a), where an LLM coordinates human-defined external modules, such as news analysts and price predictors [4, 10, 29]. However, such systems rely on predefined expert roles and typically route an entire market state or trading task to a few model-level experts, making it difficult to adapt expert usage across assets, information tokens, and decision fields. Internal expert routing, commonly instantiated through sparse *Mixture-of-Experts* (MoE), provides a finer-grained alternative by moving expert routing inside the language model, as shown in Fig. 1(b), in which a learned router activates only a small subset of experts for each token [14, 21, 25, 42]. However, conventional routers are trained mainly through the task loss and routing regularizers, which do not directly reveal each expert’s contribution or how unselected experts would perform. Moreover, each expert’s contribution to the trading decision may change as market conditions evolve. This leaves a central problem unresolved: *how to evaluate each expert’s contribution to trading decisions and track its changes as market conditions evolve?* This problem is non-trivial due to two key challenges.

First, *how can we quantify each expert’s contribution to a trading decision?* Conventional routers assign expert scores from token hidden states [14, 25]. These scores are only implicit proxies for expert suitability: the training objective does not directly verify whether a higher-scored expert contributes more to reducing the trading-decision loss. We therefore assess routing quality through controlled expert replacement. For each held-out decision token, we replace a selected expert with an unselected alternative while keeping the number of active experts unchanged, and measure the resulting change in the trading-decision loss. If router scores faithfully reflect expert contribution, they should rank experts consistently with these replacement gains. Instead, naive router (refers to conventional MoE models [28]) scores have a Pearson correlation [38] of only -0.015 with the measured gains, and 66.76% of decision tokens leave at least one better expert unselected. This substantial mismatch motivates a measure of expert contribution. We define counterfactual expert credit using a standard first-order approximation [34, 35]. It estimates how replacing a selected expert with an alternative would change the trading-decision loss; a higher credit means that the alternative is expected to reduce the loss more. We aggregate these credits into a token–expert matrix and analyze its structure across two pretrained MoE-LLMs [28, 36]. As shown in Table 1, the matrices exhibit a pronounced low-rank structure [13]: rank-16 reconstructions retain 74.2% and 77.9% of the credit energy

Table 1: Low-rank analysis of token–expert counterfactual credit.

MoE backbone	Credit Energy Retained			Better-than-Native Route Rate		
	$r = 1$	$r = 4$	$r = 16$	$r = 1$	$r = 4$	$r = 16$
OLMoE	0.185	0.373	0.742	0.643	0.812	0.983
DeepSeek-V2-Lite	0.117	0.359	0.779	0.734	0.875	0.986

r is the reconstruction rank. Better-than-Native Route Rate is the rate of decision tokens for which the experts selected using this compact representation have higher total credit than those selected by the original router.

and identify higher-credit experts than the native routers on over 98% of decision tokens. This finding suggests that expert suitability can be represented in a compact latent space. Motivated by this structure, we propose a *Query–Key* router. For each decision token, the router constructs a low-dimensional trading-demand query that represents the expertise required by the token under the current market context. Each expert is associated with a learnable expert key, and computes routing scores through query–key matching.

Second, *how can the router learn when its expert selection should change across market conditions?* Expert contribution is inherently market-dependent: an expert that is effective in one regime may become ineffective after a regime shift or event shock [1, 31]. As market evolves, the router must therefore learn not only which experts are currently useful, but also when an inactive expert has become a better alternative and should enter the route. Conventional sparse routing exposes the task loss only for the experts activated at each token, leaving the current contribution of inactive experts unobserved [14, 42]. In this work, we propose a *sparse expert selection update* mechanism. At each decision-value token, we sample a small number of inactive experts as challengers and compare them with the lowest-scored active expert in the current Top- k route. Their first-order relative credits estimate whether a challenger would reduce the trading-decision loss more than the lowest-scored active expert and should therefore replace it.

In summary, our contributions are fourfold. First, we uncover that native routers often fail to select the experts that best reduce the trading-decision loss, and token–expert counterfactual credits exhibit a pronounced approximately low-rank structure. Second, we propose TradingMoE, a trading-oriented sparse internal expert routing MoE that combines query-key router for modeling expert contribution with a sparse expert selection update mechanism for adapting the route as market context changes. Third, we provide theoretical proof for the proposed expert selection update mechanism, showing that inactive-expert sampling yields an unbiased

estimate of the update over all inactive experts and that the routing-margin update is consistent with the local loss reduction induced by expert replacement. Fourth, extensive experiments on stock, crypto, and rolling paper-trading demonstrate substantial improvements of TradingMoE over 22 baselines.

2 Related Work

Conventional AI methods for trading. Traditional machine-learning methods for trading typically rely on structured market signals, such as price-volume features and technical indicators to predict future returns, movement directions, or stock rankings [15, 47]. Deep learning methods further improve trading models by learning temporal patterns, cross-sectional dependencies, and market-aware representations from financial data [26, 55]. Reinforcement learning approaches formulates trading as a sequential decision-making problem and optimize trading actions or portfolio weights under transaction costs and risk-aware objectives [22, 30, 47, 49, 58]. However, these approaches are usually built on fixed numerical inputs and specialized task objectives, limiting their ability to incorporate heterogeneous financial information.

LLMs-based methods for trading. Recent studies adapt large language models to financial scenarios, including financial text understanding, document analysis, numerical reasoning and market forecasting [5, 6, 11, 17, 37, 44, 48, 51, 56, 57, 60]. More recent LLM-based trading systems move toward direct decision-making through agentic workflows, where LLMs act as analysts, planners, or policy agents in trading pipelines [52, 53, 59]. However, these systems typically rely on a single dense model or a fixed agentic workflows, providing limited flexibility in how specialized capabilities are invoked during decision generation.

Expert routing for trading. External expert routing decomposes trading analysis and decision-making into human-defined expert modules and coordinates them at the task or model level [4, 10, 29]. While this paradigm integrates heterogeneous expertise, its predefined expert roles and coarse routing granularity limit adaptation across assets and decision components. Internal expert routing instead performs token-level expert selection within Transformer layers, enabling finer-grained specialization [14, 21, 25, 42]. However, conventional internal routers score experts from token hidden states without explicit supervision on whether an unselected expert would reduce the trading-decision loss. Our work addresses this gap by designing and supervising internal expert routing for direct generation of trading decisions.

3 Preliminaries

Daily trading task formulation. We formulate trading task as a decision-generation problem. For each trading day n , let τ denote the predefined decision time. The model uses only market information available before τ to generate the trading decision for that day. For notational simplicity, we omit the day index n in the following. The input is a set of candidate assets $\mathcal{X} = \{x_1, x_2, \dots\}$. Asset i is represented as a structured sequence x_i comprising its sector, market capitalization, five-day open, high, low, close, and volume (OHLCV) [16] history, indicators [50], and timestamped asset-related news. Details of the construction of the input is shown in Appendix B.2. $\mathcal{X}$ is fed into a trading model, which generates a

set of trading decisions $\mathcal{Y} = \{y_1, y_2, \dots, y_M\}$. For asset i , the decision is represented as $y_i = (a_i, q_i)$, where $a_i \in \{\text{long, short, hold}\}$ denotes the trading action, and $q_i \geq 0$ denotes the corresponding position size. A long action allocates q_i to a long position, whereas a short action allocates q_i to a short position. For a hold action, q_i preserves the current position rather than opening a new position. **Internal expert routing.** Here we introduce a conventional internal expert routing architecture [14, 42]. Specifically, the market input $\mathcal{X}$ is first serialized, tokenized, and mapped to an embedding matrix:

$$\mathbf{H}^0 = \text{Embed}(\text{Tokenizer}(\text{Serialize}(\mathcal{X}))). \quad (1)$$

where $\text{Serialize}(\cdot)$ converts the structured multi-asset market state into the textual sequence. $\text{Tokenizer}(\cdot)$ denotes the tokenizer of a language model, which converts the market input into a sequence of T tokens. $\text{Embed}(\cdot)$ is the token-embedding layer, mapping each token to a d -dimensional embedding. $\mathbf{H}^0$ serves as the input for the transformer. Consider a routed Transformer layer ℓ with input $\mathbf{H}^{\ell-1} \in \mathbb{R}^{T \times d}$. Before expert routing, the attention sub-layer computes

$$\bar{\mathbf{H}}^\ell = \mathbf{H}^{\ell-1} + \text{Attn}^\ell(\text{LN}^\ell(\mathbf{H}^{\ell-1})), \quad (2)$$

where $\text{Attn}^\ell(\cdot)$ denotes the multi-head self-attention module and LN^ℓ denotes the corresponding layer-normalization module. Let $\bar{\mathbf{h}}_t^\ell$ denote the t -th row of $\bar{\mathbf{H}}^\ell$. At layer ℓ , the routing module contains E lightweight low-rank experts [20, 46]. All experts share a trainable down-projection $\mathbf{A}^\ell \in \mathbb{R}^{d_{\text{lr}} \times d}$, while each expert i has an expert-specific up-projection $\mathbf{B}_i^\ell \in \mathbb{R}^{d \times d_{\text{lr}}}$. The output of expert i for token t is defined as

$$e_i^\ell(\bar{\mathbf{h}}_t^\ell) = \frac{\lambda_{\text{lr}}}{d_{\text{lr}}} \mathbf{B}_i^\ell \mathbf{A}^\ell \bar{\mathbf{h}}_t^\ell, \quad (3)$$

where d_{lr} denotes the low-rank dimension [20] and λ_{lr} is the corresponding scaling factor [20]. The expert parameters at layer ℓ are collected as

$$\Phi^\ell = \{\mathbf{A}^\ell, \mathbf{B}_1^\ell, \dots, \mathbf{B}_E^\ell\}. \quad (4)$$

For each token t , a layer-specific scoring multilayer perceptron produces routing scores for all experts:

$$\mathbf{s}_t^\ell = \text{Score}^\ell(\bar{\mathbf{h}}_t^\ell; \theta^\ell) \in \mathbb{R}^E, \quad (5)$$

where $\text{Score}^\ell(\cdot)$ is the scoring network at layer ℓ , with parameter θ^ℓ . The i -th element $s_{t,i}^\ell$ of $\mathbf{s}_t^\ell$ denotes the routing score assigned to expert e_i^ℓ . The router activates the K highest-scoring experts:

$$\mathcal{I}_t^\ell = \text{TopK}(\mathbf{s}_t^\ell), \quad (6)$$

where $\mathcal{I}_t^\ell \subseteq \{1, \dots, E\}$ is the index set of selected experts and k is a hyperparameter that controls the routing sparsity. The routing weights over the selected experts are computed by

$$\alpha_t^\ell = \text{Softmax}\left(\left[s_{t,i}^\ell\right]_{i \in \mathcal{I}_t^\ell} / \eta\right), \quad (7)$$

where $\text{Softmax}(\cdot)$ [2] normalizes the routing scores over the selected Top- K experts, $\eta > 0$ is the temperature parameter and $\alpha_{t,i}^\ell$ denotes the resulting weight of expert i . The selected expert outputs are aggregated into a sparse expert residual:

$$\mathbf{r}_t^\ell = \sum_{i \in \mathcal{I}_t^\ell} \alpha_{t,i}^\ell \cdot e_i^\ell(\bar{\mathbf{h}}_t^\ell). \quad (8)$$

The expert residual operate in parallel with the original feed-forward network of the Transformer layer. The output representation of token t is therefore

$$\mathbf{h}_t^\ell = \text{FFN}^\ell(\bar{\mathbf{h}}_t^\ell) + \mathbf{r}_t^\ell, \quad (9)$$

where $\text{FFN}^\ell(\cdot)$ denotes the original feed-forward network at layer ℓ . The complete output of the routed layer is $\mathbf{H}^\ell = [\mathbf{h}_1^\ell; \mathbf{h}_2^\ell; \dots; \mathbf{h}_T^\ell] \in \mathbb{R}^{T \times d}$. Let $\mathcal{L}$ denote the set of routed Transformer layers. We collect the expert parameters and scoring-network parameters across all routed layers as $\Phi = \{\Phi^\ell\}_{\ell \in \mathcal{L}}$ and $\Theta = \{\Theta^\ell\}_{\ell \in \mathcal{L}}$, respectively. The output $\mathbf{H}^\ell$ is passed to the subsequent Transformer layer, and the final hidden representations are decoded to generate the structured trading decisions $\mathcal{Y}$.

Training objective. Given a market input X , the model defines an autoregressive distribution $p_{\Phi, \Theta}(\mathcal{Y} | X)$ over structured trading decisions, where Φ and Θ denote the trainable expert and router parameters, respectively. Let $\mathcal{Y}^*$ denote the corresponding ground-truth trading decision, constructed as described in Appendix B.1.

The training objective minimizes the negative log-likelihood of the ground-truth decision:

$$\mathcal{L}_{\text{task}}(\Phi, \Theta) = -\log p_{\Phi, \Theta}(\mathcal{Y}^* | X). \quad (10)$$

To compute this objective autoregressively, we serialize and tokenize $\mathcal{Y}^*$ into

$$\mathbf{z}^* = \text{Tokenizer}(\text{Serialize}(\mathcal{Y}^*)) = (z_1^*, \dots, z_{|\mathcal{Y}^*|}^*). \quad (11)$$

Under teacher forcing, the preceding ground-truth tokens $\mathbf{z}_{<j}^* = (z_1^*, \dots, z_{j-1}^*)$ are provided when predicting z_j^* . Accordingly, Eq. 10 is computed as

$$\mathcal{L}_{\text{task}}(\Phi, \Theta) = -\sum_{j=1}^{|\mathcal{Y}^*|} \log p_{\Phi, \Theta}(z_j^* | X, \mathbf{z}_{<j}^*). \quad (12)$$

Training minimizes this loss with respect to Φ and Θ , while the parameters of the pretrained language-model backbone remain frozen.

4 Proposed Method

4.1 Overview

We illustrate the overall framework of TradingMoE in Fig. 2. As shown in Fig. 2(a), the structured market state is processed by a frozen LLM backbone whose routed Transformer layers are augmented with token-level sparse residual experts. The final hidden representations are decoded into structured trading decisions. TradingMoE contains two key components. The Query-Key router in Fig. 2(b) maps each post-attention token representation to a low-dimensional trading-demand query and matches it with learnable expert keys to select the Top- k experts. The sparse expert selection update in Fig. 2(c) samples inactive experts, compares them with the lowest-scored selected expert, and uses their replacement information to update the router within the same backward pass.

4.2 Query-Key Routing

The low-rank structure of token-expert credit suggests that expert suitability can be captured through low-dimensional interactions between token requirements and expert representations. Accordingly,

we introduce a Query-Key router to parameterize token-expert routing scores.

Low-rank query-key matching. At routed layer ℓ , the router takes the post-attention representation $\bar{\mathbf{h}}_t^\ell \in \mathbb{R}^d$ of token t as input. A layer-specific query network maps it to a low-dimensional trading-demand query:

$$\mathbf{q}_t^\ell = f^\ell(\bar{\mathbf{h}}_t^\ell; \theta^\ell) \in \mathbb{R}^{d_q}, \quad (13)$$

where f^ℓ denotes the query network, implemented as a MLP. θ^ℓ denotes its parameters, and d_q is the routing dimension. The query is conditioned on both the current token and the market context encoded in its hidden representation, and is intended to capture the expertise required for processing that token. Each expert i at layer ℓ is associated with a learnable key $\mathbf{u}_i^\ell \in \mathbb{R}^{d_q}$. The routing score between token t and expert i is computed as

$$s_{t,i}^\ell = (\mathbf{q}_t^\ell)^\top \mathbf{u}_i^\ell. \quad (14)$$

A higher score indicates a stronger match between the expertise required by the token and the corresponding expert.

To make the low-rank parameterization explicit, let $\mathbf{Q}^\ell \in \mathbb{R}^{T \times d_q}$ collect the queries of all tokens and $\mathbf{U}^\ell \in \mathbb{R}^{E \times d_q}$ collect the keys of all experts. The complete query-key score matrix is

$$\mathbf{S}^\ell = \mathbf{Q}^\ell (\mathbf{U}^\ell)^\top \in \mathbb{R}^{T \times E}. \quad (15)$$

Therefore, $\text{rank}(\mathbf{S}^\ell) \leq d_q$, providing a compact parameterization consistent with the observed low-rank credit structure.

Experts routing and aggregation. The router then selects the K highest-scoring experts $\hat{\mathcal{I}}_t^\ell$ and computes their normalized routing weights α_i^ℓ following Eqs. 6 and 7, respectively. Each selected expert applies its low-rank transformation to the post-attention token representation $\bar{\mathbf{h}}_t^\ell$, as defined in Eq. 3. The resulting expert outputs are weighted by α_i^ℓ and aggregated into the sparse residual $\mathbf{r}_t^\ell$ according to Eq. 8. This residual is combined with the output of the frozen feed-forward network and the post-attention representation to produce the final token representation $\mathbf{h}_t^\ell$, as defined in Eq. 9. Collecting all token representations yields the routed-layer output $\mathbf{H}^\ell$, which is passed to the subsequent Transformer layer.

4.3 Sparse Expert Selection Update

In naive top- k routing, the task loss backpropagates only through the selected expert outputs and their routing scores. Inactive experts therefore receive no task-dependent signal indicating whether their routing scores should be increased. To address this issue, we propose a sparse expert selection update mechanism.

Detached routing margin. We first randomly sample m candidates from inactive experts, denote their index set by $\hat{\mathcal{I}}_t^\ell$. Then we define the lowest-scored expert in the current top- k route as $e_{\text{low}}^\ell(\cdot)$. For each sampled inactive expert $i \in \hat{\mathcal{I}}_t^\ell$, we compare its routing score with that of $e_{\text{low}}^\ell(\cdot)$. To prevent the resulting supervision from propagating to the attention sub-layer, we compute the score margin using a detached token representation:

$$\Delta s_{t,i}^\ell = f^\ell(\text{stopgrad}(\bar{\mathbf{h}}_t^\ell); \theta^\ell)^\top (\mathbf{u}_i^\ell - \mathbf{u}_{\text{low}}^\ell), \quad (16)$$

where $\text{stopgrad}(\cdot)$ preserves its input during the forward pass but stops gradient propagation during backpropagation. $\mathbf{u}_{\text{low}}^\ell$ denotes

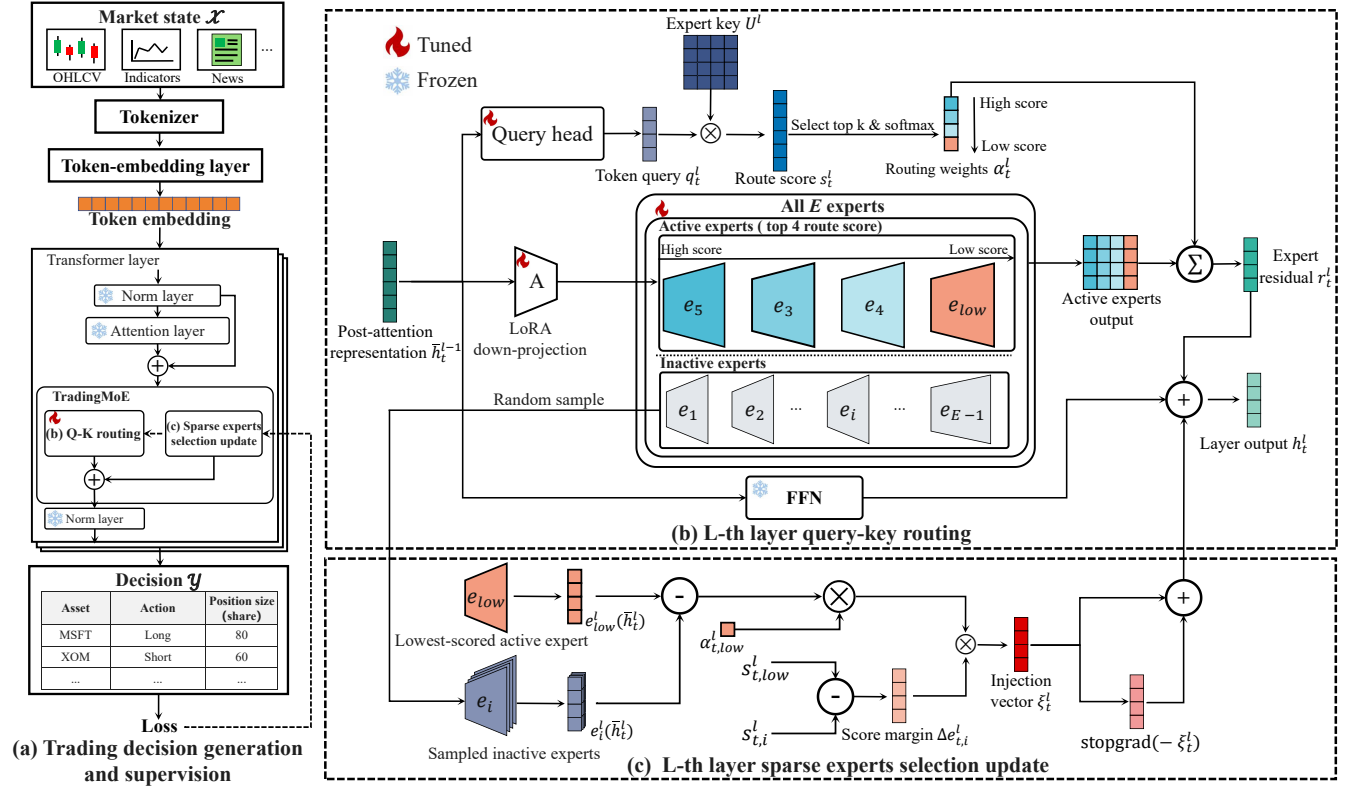


Figure 2: Overview of TradingMoE.

the key of $e_{low}^l(\cdot)$. A positive $\Delta s_{t,i}^l$ indicates that the sampled inactive expert receives a higher routing score than the lowest-scored active expert.

Same-step router update. We combine the detached routing margins with the corresponding expert-output differences:

$$\xi_t^\ell = \frac{\lambda_{sel}}{m} \sum_{i \in \hat{I}_t^\ell} \Delta s_{t,i}^\ell \text{stopgrad}(\alpha_{t,low}^\ell [e_i^\ell(\bar{\mathbf{h}}_t^\ell) - e_{low}^\ell(\bar{\mathbf{h}}_t^\ell)]), \quad (17)$$

where $\lambda_{sel} > 0$ controls the strength of the expert selection update. We inject this information into the routed-layer output through

$$\mathbf{h}_t^\ell = \text{FFN}^\ell(\bar{\mathbf{h}}_t^\ell) + \mathbf{r}_t^\ell + \xi_t^\ell - \text{stopgrad}(\xi_t^\ell). \quad (18)$$

As Eqs. 17 and 18 cancel in the forward pass, the model prediction and the task loss in Eq. 12 remain unchanged. During backpropagation, however, gradients through ξ_t^ℓ update the query-network parameters Θ and expert-key matrices $\mathbf{U} = \{\mathbf{U}^\ell\}$, including the keys of sampled inactive experts. Meanwhile, the selected expert parameters Φ are updated through the original routed branch. Training therefore minimizes $\mathcal{L}_{task}(\Phi, \Theta, \mathbf{U})$ with respect to Φ , Θ , and $\mathbf{U}$, while keeping the pretrained backbone frozen.

4.4 Theoretical Justification

We theoretically prove two properties of the sparse expert selection mechanism. First, sampling a small number of inactive experts provides an unbiased estimate of the update averaged over all inactive experts. Second, the router update adjusts the routing margin

according to whether replacing the lowest-scored selected expert with a sampled inactive expert would locally reduce the task loss.

PROPOSITION 1 (UNBIASED INACTIVE-EXPERT SAMPLING). Let $\psi^\ell = \{\theta^\ell, \mathbf{U}^\ell\}$ denote the router parameters at layer ℓ , and let $\mathbf{v}_{t,i}^\ell$ denote the gradient contribution to ψ^ℓ obtained by evaluating inactive expert i . If $\hat{I}_t^\ell$ contains m experts sampled uniformly without replacement from $(I_t^\ell)^c$, then

$$\mathbb{E}_{\hat{I}_t^\ell} \left[\frac{1}{m} \sum_{i \in \hat{I}_t^\ell} \mathbf{v}_{t,i}^\ell \right] = \frac{1}{E-k} \sum_{i \in (I_t^\ell)^c} \mathbf{v}_{t,i}^\ell. \quad (19)$$

Therefore, the sampled router-update gradient is an unbiased estimate of the average gradient over all inactive experts.

Sampling reduces the additional expert computation from $O(E-k)$ to $O(m)$. The proof is provided in Appendix A.1.

PROPOSITION 2 (REPLACEMENT-CONSISTENT ROUTING-MARGIN UPDATE). For each sampled inactive expert $i \in \hat{I}_t^\ell$, define the expert-output difference used in Eq. 17 as

$$\Delta \mathbf{e}_{t,i}^\ell = \alpha_{t,low}^\ell [e_i^\ell(\bar{\mathbf{h}}_t^\ell) - e_{low}^\ell(\bar{\mathbf{h}}_t^\ell)]. \quad (20)$$

The same-step router update induces

$$\frac{\partial \mathcal{L}_{task}}{\partial \Delta s_{t,i}^\ell} \bigg|_{sel} = \frac{\lambda_{sel}}{m} \left\langle \frac{\partial \mathcal{L}_{task}}{\partial \mathbf{h}_t^\ell}, \Delta \mathbf{e}_{t,i}^\ell \right\rangle. \quad (21)$$

Let $G_{t,i}^\ell(\rho)$ denote the reduction in task loss obtained by perturbing the routed-layer output along $\Delta \mathbf{e}_{t,i}^\ell$:

$$G_{t,i}^\ell(\rho) = \mathcal{L}_{\text{task}}(\mathbf{h}_t^\ell) - \mathcal{L}_{\text{task}}(\mathbf{h}_t^\ell + \rho \Delta \mathbf{e}_{t,i}^\ell). \quad (22)$$

If the task loss is twice differentiable in a local neighborhood, then

$$G_{t,i}^\ell(\rho) = -\rho \left\langle \frac{\partial \mathcal{L}_{\text{task}}}{\partial \mathbf{h}_t^\ell}, \Delta \mathbf{e}_{t,i}^\ell \right\rangle + O\left(\rho^2 \|\Delta \mathbf{e}_{t,i}^\ell\|^2\right). \quad (23)$$

Consequently, if replacing $\mathbf{e}_{\text{low}}^\ell(\cdot)$ with $\mathbf{e}_i^\ell(\cdot)$ is locally expected to reduce the task loss, gradient descent increases $\Delta \mathbf{e}_{t,i}^\ell$; otherwise, it decreases the routing margin.

Note that the gradient in Eq. 21 is proportional to the first-order counterfactual expert credit used in the diagnostic analysis of Sect. 1. Thus, the same-step update implicitly converts this credit into supervision on the routing margin without introducing an additional credit loss.

The proof is provided in Appendix A.2.

5 Experiments

In this section, we evaluate TradingMoE on stock and cryptocurrency markets and further assess its prospective performance through rolling paper trading.

5.1 Experimental Setup

Datasets. We evaluate TradingMoE on two multi-asset daily trading benchmarks. *Stock* is derived from FNSPID [12], a large-scale dataset containing 29.7 million stock-price records and 15.7 million time-aligned financial news articles. We select 33 U.S. equities spanning 11 sectors and use data from 2021-01-01 to 2023-12-31, which are chronologically split into training, validation, and test sets at a ratio of 7:1:2, thereby preserving temporal order. *Crypto* is a self-collected benchmark covering a 10 cryptocurrencies. Its evaluation period runs from 2025-01-01 to 2025-12-31, encompassing both bullish and bearish market phases. Details of the datasets is provided in Appendix B.2.

Baselines. We compare TradingMoE with 22 baselines from seven families, together with two passive buy and hold benchmarks. (1) *Tree-based forecasting methods*, including LightGBM[23] and XGBoost[3], construct additive ensembles of decision trees that progressively correct residual errors and capture nonlinear interactions among structured market features. (2) *Neural forecasting methods*, including LSTM [18], GRU [7, 8], MLP [19], TRA [27], and HIST [55], learn continuous representations of market data to capture temporal dependencies, cross-asset relations, and changing market patterns for return prediction. (3) *Reinforcement learning trading methods*, including FinRL-DQN, FinRL-A2C, and FinRL-PPO [30], learn portfolio policies by optimizing sequential trading rewards. (4) *Financial LLM*, including FinGPT [56], FinCast [60], and Kronos[43], adapt LLMs to financial text and market context through domain-specific pretraining or fine-tuning. (5) *General LLM*, including DeepSeek V4 Pro[54], DeepSeek V4 Flash[54], and Qwen3.6-35B-A3B[45], directly generate structured trading decisions from the market context without task-specific fine-tuning. (6) *LLM-based trading agents*, including FinAgent [59] and Trading Agent [52], use multi-step reasoning, reflection, memory, or tool-assisted analysis to produce

trading decisions. (7) *External expert-routing methods*, including LL-MoE [29], FLAG-Trader [53], TradExpert [10], and MM-DREX [4], coordinate predefined external experts to guide trading decision generation.

We additionally include two passive strategies: *Market Buy-and-Hold*, which holds the Nasdaq Composite for the stock benchmark and Bitcoin for the cryptocurrency benchmark, and *Equal-Weight Buy-and-Hold*, which allocates capital equally across all candidate assets and holds the resulting portfolio. Detailed baseline configurations are given in Appendix C.

Evaluation protocol. All model training and hyperparameter selection are performed exclusively on the training and validation splits of the Stock benchmark. The trained models are then evaluated without further training or adaptation on both the chronologically subsequent Stock test set and the held-out Crypto benchmark. For both markets, trading decisions are processed by the same portfolio simulator with identical transaction-cost and position-limit settings, as detailed in Appendix B.3. We report seven metrics, including cumulative return [33], annualized return [33], Sharpe ratio [41], maximum drawdown [32], annualized volatility [33], hit rate [39], and average daily turnover [9].

5.2 Performance Evaluation

Portfolio performance. We report the portfolio performance on Stock and Crypto in Tables 2 and 3, respectively. We make three observations. First, TradingMoE achieves the best cumulative return and Sharpe ratio on both benchmarks. On Stock, it obtains a cumulative return of 49.08%, outperforming the strongest baseline (LightGBM) by 30.89%. On Crypto, it reaches 73.79%, exceeding the strongest baseline (TRA) by 30.78%, while also achieving the lowest maximum drawdown. Second, all external expert-routing methods produce negative returns on Stock, suggesting that routing among manually defined experts may be too coarse to accommodate heterogeneous assets. Third, TradingMoE exhibits stronger robustness across markets. Competitive methods such as FinAgent, LightGBM perform well on Stock but deteriorate markedly on Crypto, whereas TradingMoE ranks first on both benchmarks.

Cumulative-return dynamics. For each benchmark, we further compares TradingMoE with the three strongest baselines ranked by cumulative return in the corresponding table, together with the market buy-and-hold strategy and show the results in Figure 3. We observe that on Stock, TradingMoE maintains a generally upward trajectory and preserves most of its accumulated gains toward the end of the evaluation period. On Crypto, although all methods exhibit substantial fluctuations, TradingMoE achieves the highest terminal return with a comparatively limited drawdown.

5.3 Ablation Study

To evaluate the contribution of each component, we compare TradingMoE with three ablated variants in Table 4. We observe that (1) Variant 2 substantially outperforms Variant 1, showing that the query-key router provides more effective token-expert matching than conventional routing. (2) Variant 3 brings only moderate improvements over Variant 1, suggesting that the spare expert selection update alone is insufficient when the underlying routing scores

Table 2: Portfolio performance on Stock.

Category	Model	Cum. Ret.↑	Ann. Ret.↑	Sharpe↑	Max DD↓	Vol.↓	Hit Rate↑	Turnover
Passive	Market Buy & Hold	+11.27%	+21.53%	1.331	12.27%	15.55%	57.97%	0.000
	Equal-Weight Buy & Hold	+2.89%	+5.34%	0.530	10.47%	10.94%	57.25%	0.000
Tree forecasting	LightGBM	<u>+18.19%</u>	<u>+35.70%</u>	<u>1.723</u>	11.09%	18.74%	54.16%	1.462
	XGBoost	+10.87%	+20.73%	1.280	8.51%	15.67%	51.82%	1.442
Neural forecasting	LSTM	+8.53%	+16.13%	1.007	7.44%	16.14%	52.17%	1.042
	GRU	+7.54%	+14.19%	0.866	9.70%	16.98%	52.17%	0.975
	MLP	+8.07%	+15.23%	0.895	12.25%	17.56%	50.72%	1.063
	TRA	+6.56%	+12.30%	0.737	10.20%	17.89%	52.90%	0.963
	HIST	+9.44%	+17.91%	1.050	8.19%	17.06%	50.72%	1.101
Reinforcement learning	FinRL-DQN	+5.93%	+11.09%	0.840	6.32%	13.62%	51.52%	0.445
	FinRL-A2C	+3.37%	+6.24%	0.560	7.20%	12.11%	49.20%	0.274
	FinRL-PPO	-12.83%	-22.18%	-1.456	14.16%	16.31%	46.73%	0.410
Financial LLM	FinGPT	-8.62%	-15.17%	-1.457	13.93%	<u>10.88%</u>	45.65%	0.014
	FinCast	-11.73%	-20.38%	-1.133	16.13%	18.57%	47.83%	1.342
	Kronos	+2.56%	+4.73%	0.376	15.47%	15.50%	50.72%	1.794
General LLM	DeepSeek V4 Pro	+12.84%	+24.69%	1.493	7.01%	15.65%	53.07%	1.050
	DeepSeek V4 Flash	+7.94%	+14.98%	1.092	<u>4.67%</u>	13.69%	52.77%	1.100
	Qwen3.6-35B-A3B	+6.82%	+12.80%	0.868	8.99%	15.28%	52.98%	1.020
LLM trading agents	FinAgent	+10.21%	+19.43%	1.562	4.57%	11.81%	53.31%	1.127
	Trading Agent	+10.42%	+19.85%	1.353	6.07%	14.12%	51.75%	1.214
External expert-routing	LLMoE	-14.45%	-24.80%	-2.285	15.99%	12.14%	47.83%	1.306
	FLAG-Trader	-6.12%	-10.89%	-0.624	9.32%	16.33%	43.48%	0.794
	TradExpert	-4.74%	-8.49%	-0.899	6.03%	9.38%	49.28%	0.444
	MM-DREX	-2.45%	-4.43%	-0.247	9.82%	14.21%	52.17%	0.914
Ours	TradingMoE	+49.08%	+107.33%	5.091	5.96%	14.55%	<u>57.91%</u>	1.298

The best result is shown in **bold**, and the second-best is underlined.

are not sufficiently informative. (3) The complete model clearly outperforms both single-component variants, demonstrating that the query-key router and selection update are complementary: the former improves expert matching, while the latter further refines routing using feedback from sampled inactive experts.

5.4 Leakage-Controlled Evaluation

LLM-based financial backtests may suffer from temporal leakage when the pretraining corpus overlaps with the retrospective evaluation period. We therefore evaluate TradingMoE under two leakage-controlled settings. First, we conduct prospective stock paper trading from 2026-04-01 to 2026-6-30 using Qwen3.5-9B (released on 2026-03) as backbone. Second, we repeat the 2025 Crypto evaluation using Qwen2.5-7B (released on 2024-09), whose checkpoint predates the entire evaluation period.

We make two observations. First, under leakage-free evaluation, TradingMoE consistently outperforms the selected baselines in both prospective stock paper trading (Figure 4) and the Qwen2.5-7B-based Crypto evaluation (Table 5). This confirms that the strong performance of TradingMoE does not rely on access to future market outcomes during pretraining. Second, the potential temporal overlap has only a limited impact. As shown in Table 5, replacing Qwen2.5-7B with Qwen3.5-9B produces only a modest performance

difference, which is much smaller than the improvement of TradingMoE over the corresponding base LLMs and competing methods. Thus, the overall advantage primarily comes from the proposed method rather than temporal leakage.

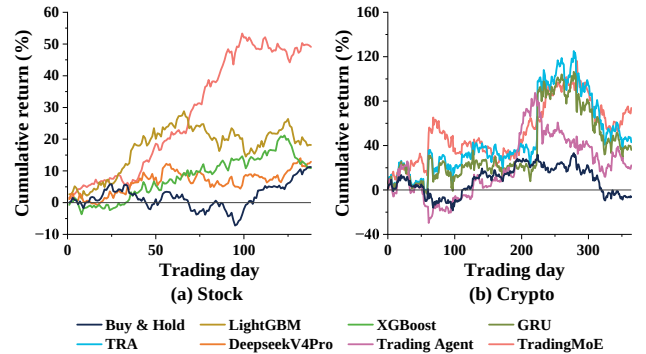


Figure 3: Cumulative return curves on Stock and Crypto.

5.5 Additional Analyses

We conduct further analyses, including hyperparameter sensitivity in Appendix E.1, multi-seed statistical robustness Appendix E.2, and transaction-cost sensitivity Appendix E.3.

Table 3: Portfolio performance on Crypto.

Category	Model	Cum. Ret.↑	Ann. Ret.↑	Sharpe↑	Max DD↓	Vol.↓	Hit Rate↑	Turnover
Passive	Market Buy & Hold	-6.38%	-6.38%	0.050	32.03%	41.72%	50.14%	0.000
	Equal-Weight Buy & Hold	-15.76%	-15.76%	0.055	44.97%	64.30%	<u>52.33%</u>	0.000
Tree forecasting	LightGBM	-2.29%	-2.29%	0.291	46.41%	66.54%	51.29%	1.398
	XGBoost	-0.42%	-0.42%	0.306	54.06%	62.28%	50.58%	1.369
Neural forecasting	LSTM	-24.22%	-24.22%	-0.083	45.39%	67.36%	48.30%	0.804
	GRU	+36.08%	+36.08%	0.758	37.39%	77.24%	48.79%	0.822
	MLP	-33.09%	-33.09%	-0.317	45.81%	63.20%	49.01%	0.738
	TRA	<u>+43.01%</u>	<u>+43.01%</u>	<u>0.838</u>	39.00%	72.00%	49.07%	0.802
	HIST	-44.61%	-44.61%	-0.551	59.39%	66.59%	50.55%	1.191
RL	FinRL-DQN	-11.47%	-11.47%	0.044	36.08%	53.70%	48.57%	0.588
	FinRL-A2C	-38.91%	-38.91%	-1.293	45.39%	33.40%	50.19%	0.096
	FinRL-PPO	-30.39%	-30.39%	-0.452	40.74%	51.44%	47.89%	0.217
Financial LMs	FinGPT	+17.75%	+17.75%	0.570	41.15%	64.88%	48.55%	0.007
	FinCast	-34.65%	-34.65%	-0.324	58.85%	65.24%	47.40%	1.159
	Kronos	-62.67%	-62.67%	-1.487	63.75%	55.71%	49.37%	1.500
General LLMs	DeepSeek V4 Pro	-19.68%	-19.68%	-0.065	50.16%	59.72%	48.75%	0.832
	DeepSeek V4 Flash	-30.61%	-30.61%	-0.287	48.15%	61.26%	48.07%	0.667
	Qwen3.6-35B-A3B	-58.43%	-58.43%	-1.299	64.32%	55.42%	46.32%	0.910
LLM trading agents	FinAgent	-50.56%	-50.56%	-0.944	58.61%	57.11%	47.61%	1.159
	Trading Agent	+22.15%	+22.15%	0.638	44.15%	58.46%	50.23%	1.195
External expert-routing	LLMoE	+18.48%	+18.48%	0.625	31.18%	39.60%	52.93%	0.434
	FLAG-Trader	-70.41%	-70.41%	-1.043	78.64%	73.89%	52.06%	0.504
	TradExpert	+15.55%	+15.55%	0.537	36.96%	56.84%	49.59%	0.409
	MM-DREX	+5.66%	+5.66%	0.366	33.04%	51.71%	<u>52.33%</u>	0.331
Ours	TradingMoE	+73.79%	+73.79%	1.355	30.14%	49.65%	51.36%	1.074

Table 4: Ablation study of key components.

Model	Q-K Router	Selection Update	Cum. Ret.↑	Sharpe↑	Max DD↓	Hit Rate↑
Variant 1	×	×	+2.83%	0.519	8.37%	53.62%
Variant 2	✓	×	+23.39%	2.919	5.52%	53.62%
Variant 3	×	✓	+5.26%	0.754	7.92%	50.72%
TradingMoE	✓	✓	+49.08%	5.091	5.96%	57.91%

✓ indicates that a component is included, and × indicates that it is excluded. Q-K router represents the query-key router.

Table 5: Crypto performance under different backbone temporal settings.

Category	Model	Cum. Ret.↑	Ann. Ret.↑	Sharpe↑	Max DD↓	Vol.↓	Hit Rate↑	Turnover
Passive	Market Buy & Hold	-6.38%	-6.38%	0.050	32.03%	41.72%	50.14%	0.000
	Equal-Weight Buy & Hold	-15.76%	-15.76%	0.055	44.97%	64.30%	<u>52.33%</u>	0.000
Selected baselines	TRA	+43.01%	+43.01%	0.838	39.00%	72.00%	49.07%	0.802
	LLMoE	+18.48%	+18.48%	0.625	31.18%	<u>39.60%</u>	52.93%	0.434
	FinGPT	+17.75%	+17.75%	0.570	41.15%	64.88%	48.55%	0.007
Base LLMs	Qwen2.5-7B Base	-3.86%	-3.86%	-0.067	23.69%	22.28%	48.22%	0.387
	Qwen3.5-9B Base	-4.82%	-4.82%	0.089	<u>25.35%</u>	41.30%	50.41%	0.661
Ours	TradingMoE (Qwen2.5-7B)	<u>+68.38%</u>	<u>+68.38%</u>	<u>1.259</u>	26.82%	51.88%	50.96%	0.739
	TradingMoE (Qwen3.5-9B)	+73.79%	+73.79%	1.355	30.14%	49.65%	<u>51.36%</u>	1.074

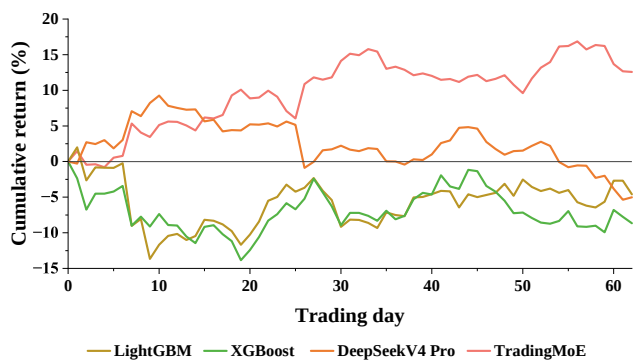


Figure 4: Cumulative return in the paper-trading.

6 Conclusions

In this paper, we proposed TradingMoE, a trading-oriented sparse internal expert routing framework for direct trading-decision generation with LLMs. Motivated by the compact structure of counterfactual expert contribution, TradingMoE employs a query-key router to match token-specific trading demands with learnable expert representations in a low-dimensional space. Its sparse expert selection update evaluates a small number of inactive experts against the lowest-scored active expert, allowing the router to revise its Top- k selection as market conditions evolve while retaining sparse computation. Extensive experiments on stock and cryptocurrency markets, together with rolling paper trading, demonstrate that TradingMoE consistently outperforms 22 baselines and generalizes effectively across different market settings.

References

- [1] Andrew Ang and Allan Timmermann. 2012. Regime changes and financial markets. *Annu. Rev. Financ. Econ.* 4, 1 (2012), 313–337.
- [2] John S Bridle. 1990. Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In *Neuro-computing: Algorithms, architectures and applications*. Springer, 227–236.
- [3] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *KDD*. 785–794.
- [4] Yang Chen, Yueheng Jiang, Zhaozhao Ma, Yuchen Cao, Jacky Keung, Kun Kuang, Leilei Gan, Yiquan Wu, and Fei Wu. 2025. MM-DREX: Multimodal-Driven Dynamic Routing of LLM Experts for Financial Trading. *arXiv preprint arXiv:2509.05080* (2025).
- [5] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan R Routledge, et al. 2021. Finqa: A dataset of numerical reasoning over financial data. In *EMNLP*. 3697–3711.
- [6] Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. 2022. Convinqa: Exploring the chain of numerical reasoning in conversational finance question answering. In *EMNLP*. 6279–6292.
- [7] Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *EMNLP*. 1724–1734.
- [8] Junyoung Chung, Çağlar Gulcehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555* (2014).
- [9] Victor DeMiguel, Lorenzo Garlappi, and Raman Uppal. 2009. Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy? *The review of Financial studies* 22, 5 (2009), 1915–1953.
- [10] Qianggang Ding, Haochen Shi, Jiadong Guo, and Bang Liu. 2024. Tradexpert: Revolutionizing trading with mixture of expert llms. *arXiv preprint arXiv:2411.00782* (2024).
- [11] Yujie Ding, Shuai Jia, Tianyi Ma, Bingcheng Mao, Xiuzhe Zhou, Liuliu Li, and Dongming Han. 2023. Integrating stock features and global information via large language models for enhanced stock return prediction. *arXiv preprint arXiv:2310.05627* (2023).
- [12] Zihan Dong, Xinyu Fan, and Zhiyuan Peng. 2024. Fnspid: A comprehensive financial news dataset in time series. In *KDD*. 4918–4927.
- [13] Carl Eckart and Gale Young. 1936. The approximation of one matrix by another of lower rank. *Psychometrika* 1, 3 (1936), 211–218.
- [14] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [15] Fuli Feng, Xiangnan He, Xiang Wang, Cheng Luo, Yiqun Liu, and Tat-Seng Chua. 2019. Temporal Relational Ranking for Stock Prediction. *TOIS* 37, 2 (2019), 1–30. doi:10.1145/3309547
- [16] Mark B Garman and Michael J Klass. 1980. On the estimation of security price volatilities from historical data. *Journal of business* (1980), 67–78.
- [17] Tian Guo and Emmanuel Hauptmann. 2024. Fine-Tuning Large Language Models for Stock Return Prediction Using Newsflow. In *EMNLP Industry Track*. 1028–1045.
- [18] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* (1997).
- [19] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. 1989. Multilayer feed-forward networks are universal approximators. *Neural networks* 2, 5 (1989), 359–366.
- [20] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [21] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [22] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. 2017. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059* (2017).
- [23] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *NeurIPS* 30 (2017).
- [24] Oliver Ledoit and Michael Wolf. 2008. Robust performance hypothesis testing with the Sharpe ratio. *Journal of Empirical Finance* 15, 5 (2008), 850–859.
- [25] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *ICLR*.
- [26] Tong Li, Zhaoyang Liu, Yanyan Shen, Xue Wang, Haokun Chen, and Sen Huang. 2024. Master: Market-guided stock transformer for stock price forecasting. In *AAAI*, Vol. 38. 162–170.
- [27] Hengxu Lin, Dong Zhou, Weiqing Liu, and Jiang Bian. 2021. Learning multiple stock trading patterns with temporal routing adaptor and optimal transport. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 1017–1026.
- [28] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
- [29] Kuan-Ming Liu and Ming-Chih Lo. 2025. LLM-Based Routing in Mixture of Experts: A Novel Framework for Trading. *arXiv preprint arXiv:2501.09636* (2025).
- [30] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. 2021. FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the second ACM international conference on AI in finance*. 1–9.
- [31] Andrew W Lo. 2004. The adaptive markets hypothesis: Market efficiency from an evolutionary perspective. *Journal of Portfolio Management*, Forthcoming (2004).
- [32] Malik Magdon-Ismael, Amir F Atiya, Amrit Pratap, and Yaser S Abu-Mostafa. 2004. On the maximum drawdown of a Brownian motion. *Journal of applied probability* 41, 1 (2004), 147–161.
- [33] Harry Markowitz. 1952. Portfolio Selection. *The Journal of Finance* 7, 1 (1952), 77–91.
- [34] Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? *NeurIPS* 32 (2019).
- [35] Pavlo Molchanov, Arun Mallya, Stephen Tyree, Iuri Frosio, and Jan Kautz. 2019. Importance estimation for neural network pruning. In *CVPR*. 11264–11272.
- [36] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. 2025. Olmo: Open mixture-of-experts language models. In *ICLR*, Vol. 2025. 62061–62121.
- [37] Rajdeep Mukherjee, Abhinav Bohra, Akash Banerjee, Soumya Sharma, Manjunath Hegde, Afreen Shaikh, Shivani Shrivastava, Koustuv Dasgupta, Niloy Ganguly, Saptarshi Ghosh, et al. 2022. Ectsum: A new benchmark dataset for bullet point summarization of long earnings call transcripts. In *EMNLP*. 10893–10906.
- [38] Karl Pearson. 1895. VII. Note on regression and inheritance in the case of two parents. *proceedings of the royal society of London* 58, 347–352 (1895), 240–242.
- [39] M Hashem Pesaran and Allan Timmermann. 1992. A simple nonparametric test of predictive performance. *Journal of Business & Economic Statistics* 10, 4 (1992), 461–465.
- [40] Dimitris N Politis and Joseph P Romano. 1994. The stationary bootstrap. *Journal of the American Statistical association* 89, 428 (1994), 1303–1313.
- [41] William F Sharpe. 1966. Mutual fund performance. *The Journal of business* 39, 1 (1966), 119–138.
- [42] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *ICLR*.
- [43] Yu Shi, Zongliang Fu, Shuo Chen, Bohan Zhao, Wei Xu, Changshui Zhang, and Jian Li. 2026. Kronos: A foundation model for the language of financial markets. In *AAAI*, Vol. 40. 25366–25373.
- [44] Yixuan Tang, Yi Yang, Allen H Huang, Andy Tam, and Justin Z Tang. 2023. FinEntity: Entity-level Sentiment Classification for Financial Texts. In *EMNLP*. 15465–15471.
- [45] Qwen Team. 2026. Qwen3. 6-35B-A3B: Agentic coding power, now open to all.
- [46] Chunlin Tian, Zhan Shi, Zhijiang Guo, Li Li, and Chengzhong Xu. 2024. Hydralora: An asymmetric lora architecture for efficient fine-tuning. *NeurIPS* 37 (2024), 9565–9584.
- [47] Jingyuan Wang, Yang Zhang, Ke Tang, Junjie Wu, and Zhang Xiong. 2019. Al-phastock: A buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks. In *KDD*. 1900–1908.
- [48] Meiyun Wang, Kiyoshi Izumi, and Hiroki Sakaji. 2024. LLMFactor: Extracting Profitable Factors through Prompts for Explainable Stock Movement Prediction. In *ACL Findings*. 3120–3131.
- [49] Rundong Wang, Hongxin Wei, Bo An, Zhouyan Feng, and Jun Yao. 2021. Commission fee is not enough: A hierarchical reinforced framework for portfolio management. In *AAAI*, Vol. 35. 626–633.
- [50] J Welles Wilder. 1978. *New concepts in technical trading systems*. Greensboro, NC.
- [51] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhjyoti Kambhadi, David Rosenberg, and Gideon Mann. 2023. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564* (2023).
- [52] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. 2024. TradingAgents: Multi-Agents LLM Financial Trading Framework. *arXiv preprint arXiv:2412.20138* (2024).
- [53] Guojun Xiong, Zhiyang Deng, Keyi Wang, Yupeng Cao, Haoqiang Li, Yangyang Yu, Xueqing Peng, Mingquan Lin, Kaleb E. Smith, Xiao-Yang Liu, Jimin Huang, Sophia Ananiadou, and Qianqian Xie. 2025. FLAG-TRADER: Fusion LLM-Agent with Gradient-based Reinforcement Learning for Financial Trading. In *Findings of*

- the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria, 13921–13934. doi:10.18653/v1/2025.findings-acl.716
- [54] Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, et al. 2026. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348* (2026).
 - [55] Wentao Xu, Weiqing Liu, Lewen Wang, Yingce Xia, Jiang Bian, Jian Yin, and Tie-Yan Liu. 2021. Hist: A graph-based framework for stock trend forecasting via mining concept-oriented shared information. *arXiv preprint arXiv:2110.13716* (2021).
 - [56] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. 2023. Fingpt: Open-source financial large language models. *arXiv preprint arXiv:2306.06031* (2023).
 - [57] Linyi Yang, Ruihai Dong, Tin Lok James Ng, and Yang Xu. 2019. Leveraging BERT to Improve the FEARS Index for Stock Forecasting. In *Proceedings of the First Workshop on Financial Technology and Natural Language Processing*. Macao, China, 54–60.
 - [58] Yunan Ye, Hengzhi Pei, Boxin Wang, Pin-Yu Chen, Yada Zhu, Ju Xiao, and Bo Li. 2020. Reinforcement-learning based portfolio management with augmented asset movement prediction states. In *AAAI*, Vol. 34. 1112–1119.
 - [59] Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, Longtao Zheng, Xinrun Wang, and Bo An. 2024. A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 4314–4325. doi:10.1145/3637528.3671801
 - [60] Zhuohang Zhu, Haodong Chen, Qiang Qu, and Vera Chung. 2025. FinCast: A Foundation Model for Financial Time-Series Forecasting. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 4539–4549.

A Theoretical Analysis and Routing Diagnostics

A.1 Unbiased Inactive-Expert Sampling

PROOF OF PROPOSITION 1. Fix token t and routed layer ℓ . Let $\mathcal{N}_t^\ell = (\mathcal{I}_t^\ell)^c$ denote the inactive expert set, whose size is $E - K$. For each $i \in \mathcal{N}_t^\ell$, let $\mathbf{v}_{t,i}^\ell$ denote its fixed candidate-wise router-gradient contribution. The gradient averaged over all inactive experts is

$$\mathbf{v}_{\mathcal{N}_t^\ell}^\ell = \frac{1}{E - K} \sum_{i \in \mathcal{N}_t^\ell} \mathbf{v}_{t,i}^\ell. \quad (24)$$

Let $\hat{\mathcal{I}}_t^\ell \subset \mathcal{N}_t^\ell$ be a subset of size m sampled uniformly without replacement. The sampled estimate is

$$\hat{\mathbf{v}}_{\hat{\mathcal{I}}_t^\ell}^\ell = \frac{1}{m} \sum_{i \in \hat{\mathcal{I}}_t^\ell} \mathbf{v}_{t,i}^\ell. \quad (25)$$

For each $i \in \mathcal{N}_t^\ell$, let $I_i = \mathbf{1}[i \in \hat{\mathcal{I}}_t^\ell]$. Uniform sampling gives $\mathbb{E}[I_i] = m/(E - K)$. Therefore,

$$\begin{aligned} \mathbb{E}_{\hat{\mathcal{I}}_t^\ell} [\hat{\mathbf{v}}_{\hat{\mathcal{I}}_t^\ell}^\ell] &= \frac{1}{m} \sum_{i \in \mathcal{N}_t^\ell} \mathbb{E}[I_i] \mathbf{v}_{t,i}^\ell \\ &= \frac{1}{E - K} \sum_{i \in \mathcal{N}_t^\ell} \mathbf{v}_{t,i}^\ell = \mathbf{v}_{\mathcal{N}_t^\ell}^\ell. \end{aligned} \quad (26)$$

Hence, the router-update gradient averaged over the sampled inactive experts is an unbiased estimate of the gradient averaged over all inactive experts. $\square$

A.2 Replacement-Consistent Routing-Margin Update

PROOF OF PROPOSITION 2. For sampled inactive expert $i \in \hat{\mathcal{I}}_t^\ell$, recall the expert-output difference

$$\Delta \mathbf{e}_{t,i}^\ell = \alpha_{t,\text{low}}^\ell \left[e_i^\ell(\bar{\mathbf{h}}_t^\ell) - e_{\text{low}}^\ell(\bar{\mathbf{h}}_t^\ell) \right]. \quad (27)$$

Equation 17 uses its detached copy and can be written as

$$\xi_t^\ell = \frac{\lambda_{\text{sel}}}{m} \sum_{j \in \hat{\mathcal{I}}_t^\ell} \Delta s_{t,j}^\ell \text{stopgrad}(\Delta \mathbf{e}_{t,j}^\ell). \quad (28)$$

Therefore,

$$\frac{\partial \xi_t^\ell}{\partial \Delta s_{t,i}^\ell} = \frac{\lambda_{\text{sel}}}{m} \text{stopgrad}(\Delta \mathbf{e}_{t,i}^\ell). \quad (29)$$

Let

$$\mathbf{h}_{t,0}^\ell = \text{FFN}^\ell(\bar{\mathbf{h}}_t^\ell) + \mathbf{r}_t^\ell \quad (30)$$

denote the routed-layer output before the zero-forward injection. Equation 18 gives

$$\mathbf{h}_t^\ell = \mathbf{h}_{t,0}^\ell + \xi_t^\ell - \text{stopgrad}(\xi_t^\ell). \quad (31)$$

Since the stop-gradient term has zero derivative,

$$\left. \frac{\partial \mathbf{h}_t^\ell}{\partial \Delta s_{t,i}^\ell} \right|_{\text{sel}} = \frac{\lambda_{\text{sel}}}{m} \text{stopgrad}(\Delta \mathbf{e}_{t,i}^\ell). \quad (32)$$

Applying the chain rule and using the identical forward value of the detached copy yields

$$\left. \frac{\partial \mathcal{L}_{\text{task}}}{\partial \Delta s_{t,i}^\ell} \right|_{\text{sel}} = \frac{\lambda_{\text{sel}}}{m} \left\langle \frac{\partial \mathcal{L}_{\text{task}}}{\partial \mathbf{h}_t^\ell}, \Delta \mathbf{e}_{t,i}^\ell \right\rangle. \quad (33)$$

We next examine whether the direction of this gradient agrees with expert-replacement utility. For intervention scale $\rho > 0$, define

$$G_{t,i}^\ell(\rho) = \mathcal{L}_{\text{task}}(\mathbf{h}_t^\ell) - \mathcal{L}_{\text{task}}(\mathbf{h}_t^\ell + \rho \Delta \mathbf{e}_{t,i}^\ell). \quad (34)$$

A first-order Taylor expansion around $\mathbf{h}_t^\ell$ gives

$$G_{t,i}^\ell(\rho) = -\rho \left\langle \frac{\partial \mathcal{L}_{\text{task}}}{\partial \mathbf{h}_t^\ell}, \Delta \mathbf{e}_{t,i}^\ell \right\rangle + O(\rho^2 \|\Delta \mathbf{e}_{t,i}^\ell\|^2). \quad (35)$$

If replacing $e_{\text{low}}^\ell(\cdot)$ with $e_i^\ell(\cdot)$ is locally expected to reduce the task loss, then $G_{t,i}^\ell(\rho) > 0$ to first order and

$$\left\langle \frac{\partial \mathcal{L}_{\text{task}}}{\partial \mathbf{h}_t^\ell}, \Delta \mathbf{e}_{t,i}^\ell \right\rangle < 0. \quad (36)$$

The induced gradient with respect to $\Delta s_{t,i}^\ell$ is therefore negative, so gradient descent increases the routing margin. Conversely, a locally harmful replacement produces a positive gradient and decreases the routing margin. When the first-order effect is zero, the margin remains unchanged to first order. $\square$

A.3 Structural Properties of Boundary-Relative Credit

The inner product appearing in Proposition 2 naturally defines the boundary-relative credit

$$\Delta C_{t,i}^\ell = - \left\langle \frac{\partial \mathcal{L}_{\text{task}}}{\partial \mathbf{h}_t^\ell}, \Delta \mathbf{e}_{t,i}^\ell \right\rangle. \quad (37)$$

Let the raw credit of expert j be

$$C_{t,j}^\ell = - \left\langle \frac{\partial \mathcal{L}_{\text{task}}}{\partial \mathbf{h}_t^\ell}, e_j^\ell(\bar{\mathbf{h}}_t^\ell) \right\rangle. \quad (38)$$

Substituting the definition of $\Delta \mathbf{e}_{t,i}^\ell$ gives

$$\Delta C_{t,i}^\ell = \alpha_{t,\text{low}}^\ell (C_{t,i}^\ell - C_{t,\text{low}}^\ell). \quad (39)$$

Ordering preservation. For any two candidate experts i and j ,

$$\Delta C_{t,i}^\ell - \Delta C_{t,j}^\ell = \alpha_{t,\text{low}}^\ell (C_{t,i}^\ell - C_{t,j}^\ell). \quad (40)$$

Since $\alpha_{t,\text{low}}^\ell > 0$, boundary-relative credit preserves the within-token ordering of raw expert credit.

Rank bound. For structural analysis, extend $\Delta C_{t,i}^\ell$ to all experts and collect the raw credits across T target tokens and E experts into $\mathbf{C}^\ell \in \mathbb{R}^{T \times E}$. Let $\mathbf{c}^\ell \in \mathbb{R}^T$ contain the credit of the lowest-scored active expert for each token:

$$\mathbf{c}_t^\ell = C_{t,\text{low}_t}^\ell. \quad (41)$$

We further define

$$\mathbf{D}^\ell = \text{diag}(\alpha_{1,\text{low}_1}^\ell, \dots, \alpha_{T,\text{low}_T}^\ell). \quad (42)$$

The boundary-relative credit matrix can then be written as

$$\Delta \mathbf{C}^\ell = \mathbf{D}^\ell (\mathbf{C}^\ell - \mathbf{c}^\ell \mathbf{1}^\top). \quad (43)$$

Since left multiplication cannot increase rank and $\mathbf{c}^\ell \mathbf{1}^\top$ has rank at most one,

$$\begin{aligned} \text{rank}(\Delta \mathbf{C}^\ell) &\leq \text{rank}(\mathbf{C}^\ell - \mathbf{c}^\ell \mathbf{1}^\top) \\ &\leq \text{rank}(\mathbf{C}^\ell) + 1. \end{aligned} \quad (44)$$

Thus, using the lowest-scored active expert as the reference preserves expert ordering and introduces at most a rank-one shift to the raw credit structure.

A.4 Low-Rank Credit Diagnostic

For this diagnostic, we form a token-by-expert raw credit matrix C at every routed Transformer layer, subtract each row mean, and compute rank- r SVD reconstructions. For OLMoE-1B-7B-0125-Instruct, each matrix is 117×64 and is collected from six prompts with Top-4 routing. For DeepSeek-V2-Lite, we use four prompts of at most 96 tokens with 64 routed experts and Top-6 routing; shared experts remain on the common path and are not included as columns of C .

Credit Energy Retained is the fraction of row-centered Frobenius energy retained. *Higher-Credit Selection Rate* is the fraction of target tokens for which the Top- K experts selected from the rank- r reconstruction have higher total credit than those selected by the naive internal router. Each entry in Table 1 is averaged over all routed layers. SVD fitting and comparison use the same matrix, so this diagnostic evaluates representational capacity rather than held-out prediction.

A.5 Router-Contribution Alignment

This diagnostic examines whether internal-router scores reflect the actual task-loss reduction produced by expert replacement. For each trading-action or position-size target token t at routed layer ℓ , we identify the lowest-scored active expert $e_{\text{low}}^\ell(\cdot)$ and evaluate every inactive expert $i \notin \mathcal{I}_t^\ell$.

The router margin is $s_{t,i}^\ell - s_{t,\text{low}}^\ell$, while the realized contribution of candidate i is measured by the finite replacement gain $G_{t,i}^\ell(1)$ defined in Proposition 2. We compute their Pearson correlation over all candidate pairs in the held-out validation set, across all routed layers and multiple training checkpoints. We also report the fraction of target tokens containing at least one inactive expert with a positive replacement gain.

The Pearson correlation between router margins and replacement gains is -0.015 , while 66.76% of target tokens contain at least one beneficial inactive expert. These results show that naive internal-router scores provide little information about which experts would most reduce the trading-decision loss.

B Experimental and Evaluation Details

B.1 Ground-Truth Decision Construction

For each trading day, we compute the open-to-close return r_i of every candidate asset and select the five assets with the largest absolute returns. Ties are broken by symbol order. Assets outside the selected set are assigned

$$y_i = (\text{hold}, 0).$$

For each selected asset, the trading direction is determined by its return. A return of at least 0.2% is labeled long, a return of at most -0.2% is labeled short, and a return between these thresholds is labeled hold. For a non-hold decision, the position size q_i is determined by the absolute return, as shown in Table 6. A hold decision has $q_i = 0$.

The resulting action and position size form the ground-truth decision $y_i = (a_i, q_i)$ for each asset, and all asset-level decisions constitute the ground-truth decision set $\mathcal{Y} = \{y_1, y_2, \dots, y_M\}$.

Table 6: Position sizes based on absolute returns.

Absolute Return	Position Size q_i
$0.2\% \leq r_i \leq 0.6\%$	1
$0.6\% < r_i \leq 1.2\%$	2
$1.2\% < r_i \leq 1.8\%$	3
$1.8\% < r_i \leq 2.4\%$	4
$ r_i > 2.4\%$	5

B.2 Data and Causal Preprocessing

Table 7 gives the decision dates used by every experiment. Stock is split chronologically and is the only source of parameter updates and model selection; Crypto covers the complete 2025 calendar and is used only for out-of-domain evaluation.

Table 7: Chronological splits used in the experiments.

Market	Split	Decision dates (inclusive)	Days	Asset-day rows
Stock	Train	2021-03-31–2023-03-03	485	16,005
Stock	Validation	2023-03-06–2023-06-12	69	2,277
Stock	Test	2023-06-13–2023-12-31	138	4,554
Crypto	OOD test	2025-01-01–2025-12-31	365	3,650

Sources and universes. Stock prices and news come from FN-SPID [12]. The universe was fixed before splitting by restricting candidates to U.S.-listed FN-SPID equities with valid OHLCV and SEC total-asset metadata, while covering 11 sectors and large-, mid-, and small-size strata where available. It contains AIT, AMT, AMZN, APD, AWR, BHP, BKNG, CAT, CMC, COP, CUBE, DUK, ELF, EXEL, FOXA, FSLY, GE, GOOGL, HOG, JNJ, JPM, KHC, MSFT, NRG, NVDA, RDN, RJF, RRC, TGNA, VICI, WMT, XOM, and ZBH. Crypto combines daily OHLCV files with the CryptoCompare news archive and fixes ten USDT spot pairs before evaluation: ADA-USDT, BTC-USDT, DOGE-USDT, ETH-USDT, HBAR-USDT, LINK-USDT, LTC-USDT, OKB-USDT, TRX-USDT, and XRP-USDT.

Prices and features. FN-SPID retains raw OHLCV and adjusted close. Because the task and simulator use same-session open-to-close returns, labels use the contemporaneous raw open and close; adjusted close remains in the source record for corporate-action auditing. Rows without a valid date or close are removed, missing same-row OHLC fields fall back to close, and missing volume is set to zero; prices are never interpolated across dates. Each sample requires five completed bars and uses trailing 30/60-day indicators. Stock windows end at the preceding trading-day close, and Crypto windows end at the preceding UTC daily close.

Causal news. Stock news is sourced from the FN-SPID news corpus [12], comprising 26,305 unique asset-article records for the selected universe. At 20:00 America/New_York on the preceding day, the two most recent asset-specific records from the prior seven days are retained. Crypto uses the analogous seven-day window ending at the preceding UTC day boundary, draws from coin-specific and broad-market categories, and retains at most two records by relevance, recency, and sentiment magnitude; the title, or the first 30 words when absent, forms its summary. Empty windows produce an empty news list, and every method reads the same frozen offline records.

B.3 Unified Portfolio Construction

All active methods are evaluated through the same decision-conversion, portfolio-construction, and execution pipeline. For trading day n , the native output of each method is first converted into the canonical asset-level decision $y_{n,i} = (a_{n,i}, q_{n,i})$ defined in Section 3, where $a_{n,i} \in \{\text{long, short, hold}\}$ denotes the trading action and $q_{n,i} \in [0, 5]$ denotes the position size.

Table 8 summarizes the conversion for different output families. Direct decision generators already produce $a_{n,i}$ and $q_{n,i}$ and therefore require no additional conversion. For a return forecaster, we retain the five assets with the largest absolute predicted returns. The predicted return determines the action, and its magnitude is converted to a position size using the intervals in Table 6. For an RL policy, the signed action is clipped to $[-1, 1]$; its sign determines the action, and its magnitude determines the position size. All conversions depend only on model outputs and do not access realized returns.

Table 8: Conversion of native outputs to canonical trading decisions.

Output family	Native output	Canonical decision
Return forecaster	Predicted return $\hat{r}_{n,i}$	$a_{n,i} = \text{Dir}(\hat{r}_{n,i}), q_{n,i} = Q(\hat{r}_{n,i})$
Decision generator	Action $a_{n,i}$, position size $q_{n,i}$	$(a_{n,i}, q_{n,i})$
RL policy	Signed action $\hat{a}_{n,i}$	$a_{n,i} = \text{Dir}(\hat{a}_{n,i}), q_{n,i} = 5 \text{clip}(\hat{a}_{n,i}, -1, 1) $

For return forecasts, $\text{Dir}(z)$ returns long when $z \geq 0.2\%$, short when $z \leq -0.2\%$, and hold otherwise. For signed RL actions, it returns long when $z > 0$, short when $z < 0$, and hold when $z = 0$. $Q(\cdot)$ denotes the return-to-position-size mapping in Table 6.

Let $\mathcal{S}_n$ denote the resulting set of at most five active assets. If a method produces more than five active decisions, only the five with the largest position sizes are retained, with ticker order used to break ties. The target portfolio weight of asset i is

$$w_{n,i} = \begin{cases} 0.2 \sigma(a_{n,i}) q_{n,i} / 5, & i \in \mathcal{S}_n, \\ 0, & \text{otherwise,} \end{cases} \quad (45)$$

where $\sigma(\text{long}) = 1$, $\sigma(\text{short}) = -1$, and $\sigma(\text{hold}) = 0$. This construction limits each asset to 20% of the portfolio and ensures $\sum_i |w_{n,i}| \leq 1$. Assets that are missing, unselected, or assigned hold receive zero weight. Unknown assets are discarded, and a response containing no valid decision produces an empty portfolio.

At the beginning of each trading day, the portfolio is rebalanced from the previous target weights w_{n-1} to the current target weights w_n . The resulting turnover is

$$u_n = \sum_i |w_{n,i} - w_{n-1,i}|, \quad w_0 = \mathbf{0}. \quad (46)$$

Thus, an asset that is no longer selected is reduced to zero weight, while a newly selected asset is adjusted from zero to its target weight. For an asset retained in the portfolio, transaction costs are incurred only on the change in its target weight. Reversing a position from long to short, or from short to long, incurs turnover equal to the full difference between the two signed weights.

Let $r_{n,i}$ denote the open-to-close return of asset i on day n . The net daily portfolio return is

$$r_n^{\text{net}} = \sum_i w_{n,i} r_{n,i} - c u_n, \quad c = 0.0005, \quad (47)$$

where c is the one-way transaction-cost rate of 5 bps. The daily return is computed using the current target weights w_n , and no additional full-liquidation cost is imposed at the end of the day.

Signed weights permit both long and short positions, while the unit gross-exposure limit prevents leverage. Unallocated capital is treated as zero-return cash. Borrow fees, additional slippage, and cash interest are not modeled. The same conversion, rebalancing, and transaction-cost rules are applied to TradingMoE and all active baselines. The passive Buy&Hold benchmarks bypass this pipeline and are reported only as references.

B.4 Metrics

Let T be the number of dates, $r_t = r_t^{\text{net}}$, $V_0 = 1$, $V_t = \prod_{s=1}^t (1+r_s)$, and $A_M = 252$ for Stock and 365 for Crypto. We report cumulative return $V_T - 1$, annualized return $V_T^{A_M/T} - 1$, Sharpe ratio $\sqrt{A_M} \bar{r} / \text{std}(r)$ with zero risk-free rate, annualized volatility $\sqrt{A_M} \text{std}(r)$, and conventional maximum drawdown. For target weight $w_{t,i}$ and realized same-day open-to-close return $r_{t,i}$, Hit Rate is

$$\text{HitRate} = \frac{\sum_{t,i} \mathbf{1}[w_{t,i} r_{t,i} > 0]}{\sum_{t,i} \mathbf{1}[|w_{t,i}| > 0, |r_{t,i}| > 0]}. \quad (48)$$

Thus unselected assets, hold decisions, and zero realized returns do not enter the denominator. Average daily turnover is $T^{-1} \sum_t \tau_t$ and counts both the opening and closing legs without the conventional one-half factor.

B.5 Statistical Robustness Protocol

We retrain TradingMoE on Stock with five random seeds, which control adapter and router initialization, shadow sampling, and data ordering; all other settings match Appendix D. Table 9 reports per-seed results.

Table 9: Per-seed results on Stock against LightGBM.

Run	Cum. Ret.	Sharpe	LW p	HAC p	Return 95% CI
Seed 39	+50.05%	5.366	0.026	0.020	[+23.96%, +84.94%]
Seed 40	+54.68%	5.827	0.020	0.008	[+33.04%, +81.19%]
Seed 41	+48.09%	4.914	0.048	0.036	[+21.08%, +84.09%]
Seed 42 (ref.)	+49.08%	5.091	0.085	0.076	[+17.32%, +90.99%]
Seed 43	+45.72%	4.678	0.028	0.033	[+23.99%, +72.58%]
Seed average	+49.52%	—	0.009	0.004	[+28.51%, +75.09%]

LW and HAC are one-sided p -values against LightGBM; Section E.2 reports the corresponding two-sided values for the seed average.

Tests. All tests compare daily net return series over the common test dates against LightGBM, the strongest baseline. The Ledoit-Wolf test is the HAC-robust Sharpe-difference test of Ledoit and Wolf [24]; the HAC test is a Newey-West-adjusted t -test on the mean daily return difference with automatic lag selection. The seed-average row tests the arithmetic mean of the five daily return series, whose null hypothesis is that the expected daily excess return over LightGBM is non-positive; we do not report a Sharpe point estimate for this averaged series, since cross-seed averaging mechanically reduces volatility and would overstate the Sharpe ratio of a single deployed model.

Confidence intervals. Intervals are stationary-block-bootstrap percentile intervals [40] with expected block length 12 and 10,000

resamples. Under block lengths 10, 12, 15, and 20, the seed-average return lower bound varies only within $[+28.5\%, +29.1\%]$ and the Sharpe lower bound within $[4.70, 4.82]$.

C Further Descriptions of Baselines

We provide additional descriptions of the baseline methods used in our experiments. All baselines follow the chronological information cutoff and portfolio-execution protocol described in Section 5. Methods that require task-specific optimization are trained only on the Stock training split, while Crypto is retained as a held-out cross-market benchmark without further training or adaptation. Each method preserves its original architecture and learning objective. We standardize only its terminal interface with the trading environment: methods that directly generate trading decisions produce the decision JSON described in Section 3, whereas methods that output return forecasts or ranking scores are connected to the same forecast-to-decision conversion and portfolio simulator. All decisions are evaluated under identical transaction costs, position constraints, and daily execution frequency.

(1) Passive Benchmark.

- **Market Buy-and-Hold:** A passive strategy that maintains a fixed market exposure throughout the evaluation period. We use NASDAQ Buy-and-Hold for Stock and BTC Buy-and-Hold for Crypto.
- **Equal-Weight Buy-and-Hold:** A passive strategy that allocates capital equally across all candidate assets at the beginning of the evaluation period and holds the resulting portfolio without active rebalancing.

(2) Gradient-Boosted Tree Models. Both tree-based methods operate on the same causal tabular market features and predict future returns, which are subsequently translated into portfolio decisions using the common execution protocol.

- **LightGBM** [23]: A histogram-based gradient-boosting framework that grows trees leaf-wise and efficiently models nonlinear interactions among price, volume, technical, and portfolio-state features.
- **XGBoost** [3]: An additive tree-ensemble model that sequentially fits regression trees to the residual errors of the preceding ensemble. Regularization is applied to control tree complexity and reduce overfitting.

(3) Deep-Learning Predictors. These methods learn next-day return signals from structured market histories. Their return predictions are converted into the common action and position-size space before portfolio execution.

- **LSTM** [18]: A recurrent sequence model that encodes the historical market trajectory using gated memory cells. The final temporal representation is used to predict the future return of each asset.
- **GRU** [7, 8]: A recurrent predictor that models temporal dependencies using update and reset gates. Compared with LSTM, it adopts a more compact recurrent structure while retaining the ability to capture long-range market patterns.
- **MLP** [19]: A feed-forward neural predictor that flattens the rolling market features into a fixed-dimensional representation and maps it to asset-level future-return estimates. It

serves as a basic dense neural baseline without an explicit temporal module.

- **TRA** [27]: A temporal routing architecture that maintains multiple return predictors for different latent trading patterns. A routing adaptor assigns each sample to the relevant predictors, while an optimal-transport objective encourages the predictors to capture distinct temporal regimes.
- **HIST** [55]: A relation-aware stock predictor that models cross-asset dependencies through concept-oriented representations. It aggregates information associated with both predefined and latent market concepts to generate asset-level return forecasts.

(4) Reinforcement-Learning Trading Methods. The FinRL variants receive a structured market and portfolio state and learn trading actions through interaction with the same portfolio simulator used for evaluation.

- **FinRL-DQN** [30]: A value-based trading agent that estimates the expected long-term return of each discrete portfolio action using a deep Q-network. The action with the highest estimated value is selected at each trading step.
- **FinRL-A2C** [30]: An actor-critic trading method in which the actor generates portfolio actions and the critic estimates their state values. The advantage signal is used to jointly optimize the policy and value networks.
- **FinRL-PPO** [30]: A policy-gradient method that updates the trading policy using a clipped surrogate objective, limiting excessively large policy changes and improving optimization stability in the portfolio environment.

(5) Financial Language and Time-Series Models. These baselines use financial-domain pre-training to provide return forecasts or direct trading decisions. Forecasting outputs are passed through the shared forecast-to-decision conversion before execution.

- **FinGPT** [56]: A finance-oriented language model adapted to financial texts and instruction-following tasks. We provide the same structured trading context and use its generated response as the final decision under the common JSON schema.
- **FinCast** [60]: A foundation model for financial time-series forecasting. It produces asset-level future forecasts from historical market sequences, which are converted into the standardized trading actions and position sizes.
- **Kronos** [43]: A pre-trained financial time-series model that captures temporal patterns from market sequences and predicts future price or return dynamics. Its forecasts are evaluated through the same portfolio-construction and execution pipeline as the other forecasting baselines.

(6) Frontier LLM Baselines. These models directly receive the structured market-state prompt and generate the required decision JSON without task-specific fine-tuning. The same prompt template, temporal cutoff, asset universe, and output schema are used across all three models.

- **DeepSeek V4 Pro** [54]: The full-capability DeepSeek endpoint is prompted to analyze the supplied market context and directly generate predicted returns, actions, and position-size levels for the selected assets.

- **DeepSeek V4 Flash** [54]: A separate DeepSeek serving variant evaluated with the same input and output protocol as DeepSeek V4 Pro. It serves as an additional direct-decision LLM baseline under an identical trading environment.
- **Qwen3.6-35B-A3B** [45]: The official Qwen3.6-35B-A3B checkpoint that processes the complete trading prompt and directly produces the standardized decision JSON. No parameters are updated on the Stock or Crypto benchmark.

(7) **LLM-Based Trading Agents.** Unlike direct prompting, these methods retain their original agentic workflows, including intermediate analysis, reflection, and planning. Their final recommendations are converted into the common decision schema and evaluated by the same portfolio simulator.

- **FinAgent** [59]: A tool-augmented financial agent that combines multimodal market information with intermediate analysis and reflection. Its agent workflow produces a final trading recommendation after integrating the available market evidence.
- **Trading Agent** [52]: A role-based multi-agent trading framework in which specialized agents conduct market analysis, debate candidate strategies, assess risk, and pass their conclusions to a final trading agent for decision generation.

(8) **External Expert-Routing Methods.** These approaches coordinate human-defined external experts at the model or workflow level. We preserve the expert decomposition and routing mechanism proposed by each method, while evaluating the final decision under our common temporal and portfolio protocol.

- **LLMoE** [29]: An LLM-based mixture-of-experts framework in which a model-level router selects or combines predefined financial experts according to the current market context. The routed expert outputs are aggregated to form the final trading recommendation.
- **FLAG-Trader** [53]: A financial trading agent that combines LLM-generated market representations with gradient-based reinforcement learning. The trainable policy uses trading feedback to refine the actions derived from the LLM-agent workflow.
- **TradExpert** [10]: A trading framework composed of specialized LLM experts responsible for different analytical roles or information sources. A coordinating module integrates their outputs into a unified trading decision.
- **MM-DREX** [4]: A multimodal dynamic-routing framework that conditions expert selection on heterogeneous market signals. Its router dynamically activates or weights external LLM experts before aggregating their recommendations.

The above standardization affects only the terminal decision interface and portfolio execution. Each baseline retains its original internal architecture, expert organization, training objective, and inference workflow.

D Implementation Details

Environment. The environment in which we run experiments is:

- Operating system: Ubuntu 22.04;
- GPU information: 16 NVIDIA A800-SXM4-80GB GPUs.

Optimizer. For TradingMoE, we use the AdamW optimizer. For all trainable baselines, we retain the architectures, optimizers, learning-rate schedules, and hyperparameters specified by their official releases. We perform no method-specific search on either test set; where an official procedure selects a checkpoint, selection uses only the Stock validation split.

Details of baselines. For all open-source baselines, we use the officially provided code or a faithful interface wrapper around it, together with the released architecture and optimization settings. Only dataset-dependent input and output dimensions are adapted to the common market features, asset universe, and trading-decision space; the same portfolio postprocessor and simulator are then applied to every method.

For FinRL-DQN, FinRL-A2C, and FinRL-PPO [30], we use the official FinRL implementations. FinRL-A2C uses 5 rollout steps, a learning rate of 7×10^{-4} , and an entropy coefficient of 0.01. FinRL-PPO uses 2,048 rollout steps, a batch size of 64, a learning rate of 2.5×10^{-4} , and an entropy coefficient of 0.01. FinRL-DQN follows the default DQN configuration provided in the official implementation.

For LSTM [18] and GRU [7, 8], we employ two recurrent layers with a hidden dimension of 64 and dropout of 0. The models are trained for at most 200 epochs with a learning rate of 10^{-3} and a batch size of 800. For MLP [19], we use the official Qlib configuration with a learning rate of 2×10^{-3} , a batch size of 4,096, and at most 8,000 optimization steps. For TRA [27], we use a two-layer LSTM backbone with a hidden dimension of 64 and three latent trading states. Its routing adaptor is implemented as a one-layer LSTM with a hidden dimension of 32; the sequence length and batch size are set to 60 and 1,024, respectively. For HIST [55], we employ a two-layer LSTM backbone with a hidden dimension of 64, use MSE as the prediction loss, and set the learning rate to 10^{-4} .

For LightGBM [23] and XGBoost [3], we retain the hyperparameters in their official baseline configurations without task-specific tuning. The executable configuration files used for all reported runs are included in the supplementary material.

For Qwen3.6-35B-A3B [45], DeepSeek V4 Pro [54], and DeepSeek V4 Flash [54], we use the official identifiers Qwen/Qwen3.6-35B-A3B, deepseek-v4-pro, and deepseek-v4-flash, respectively, with default reasoning configurations without task-specific fine-tuning. All three models receive the same trading prompt and decision format as TradingMoE. For the prospective DeepSeek curve in Figure 4, we use the flagship endpoint available at each decision date: DeepSeek-V3.2 through 2026-04-23 and DeepSeek-V4-Pro from 2026-04-24 onward.

For FinAgent [59], we preserve the officially released market-intelligence, diversified-memory, reflection, and decision modules. For Trading Agent [52], we retain its original analyst roles, debate procedure, risk-management module, and default model assignments.

For LLMoE [29], we use a five-day lookback window, a Llama-3.2 router, and two sentiment-oriented experts. Each expert uses a feed-forward network with hidden dimensions of 128, 64, and 32, with dropout rates of 0.3 and 0.2 after the second and third layers. For FLAG-Trader [53], we use SmoLLM2-135M-Instruct as the backbone and set the learning rate, rollout length, discount factor, and PPO clipping coefficient to 5×10^{-4} , 40, 0.95, and 0.2, respectively. The model is trained for 13,860 steps. For TradExpert [10], we employ

four specialized experts and one general expert based on LLaMA-2-7B. LoRA rank, scaling factor, and dropout are set to 8, 16, and 0.1, respectively; the model is trained for 30 epochs with a learning rate of 10^{-4} , a batch size of 4, and 8 gradient-accumulation steps. For MM-DREX [4], we use Qwen2.5-VL-72B-Instruct with four expert heads and a 100-day lookback window. Its LoRA rank, scaling factor, dropout, and learning rate are set to 16, 32, 0.05, and 3×10^{-5} , respectively; the discount factor and policy clipping coefficient are set to 0.99 and 0.15.

For FinGPT [56], we use the officially released Llama-2-7B LoRA financial-forecaster checkpoint. For FinCast [60], we use its released pre-trained checkpoint under the recommended zero-shot forecasting setting. For Kronos [43], we use the officially released checkpoint and default inference configuration, with temperature 1.0, Top- p 0.9, and one forecast sample.

Details of TradingMoE. For our proposed TradingMoE, we freeze Qwen3.5-9B and patch every Transformer MLP block with shared-down LoRA experts. We use $r_L=12$, $\alpha_L=24$, $E=64$, $d_q=16$, Top- $k=4$, and $m=2$ sampled inactive experts. The bias-free query head is $\text{Linear}(d, d_q) - \text{GELU} - \text{Linear}(d_q, d_q)$; expert codes and up-projections are initialized from $\mathcal{N}(0, 0.02^2)$ and $\mathcal{N}(0, 10^{-6})$, respectively, while the remaining linear maps use PyTorch defaults. Router-score temperature is 1.0, selected-weight temperature is $\tau=0.02$, credit scale is $\lambda=0.05$, and the RMS floor is $\epsilon=10^{-6}$; the balance, z-loss, and entropy coefficients are all zero in the reported runs.

Training uses AdamW with learning rate 10^{-4} , $(\beta_1, \beta_2) = (0.9, 0.999)$, optimizer $\epsilon = 10^{-8}$, weight decay 0.01, no warmup or scheduler, micro-batch size one, one gradient-accumulation step, and gradient clipping at 1.0. We train for 20,000 updates in bfloat16 with DeepSpeed ZeRO-3 and eight-way sequence parallelism, use a 16,000-token training limit and seed 42, and save every 5,000 updates. We select the checkpoint with the highest Stock validation cumulative return (step 20,000, also the highest validation Sharpe) before a single test evaluation; decoding is greedy with a 32,000-token input limit and at most 200 generated tokens. The backbone remains frozen, while the shared down-projections, expert up-projections, query heads, and expert codes are updated; inference removes shadows and credit assignment and executes only ordinary Top- k routing.

One prompt is formed for each decision date and lists the complete candidate-asset universe alphabetically. Each asset entry contains the sector, market capitalization, five completed OHLCV bars, technical indicators, and timestamped news defined in Section 3. The model is instructed to generate the canonical asset-level decisions $y_i = (a_i, q_i)$. Unmentioned assets are assigned (hold, 0) to form the complete decision set $\mathcal{Y}$.

E Additional Experimental Analyses

E.1 Hyperparameter Sensitivity

We vary the number of each layer experts E and query dimension d_q while keeping all other settings fixed, as shown in Fig. 5. We observe that performance is sensitive to their joint configuration: overly large expert sets or query dimensions often degrade returns, especially on Crypto. The setting $E = 64$ and $d_q = 16$ achieves best performance on both markets, consistent with the compact token-expert structure identified in our analysis.

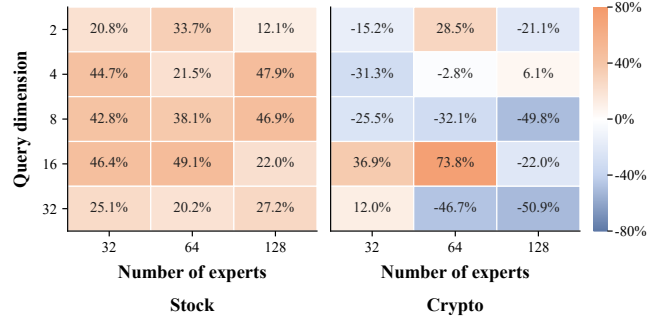


Figure 5: Impact of expert count and query dimension.

E.2 Statistical Robustness

To verify that the advantage on Stock is not an artifact of a single training run, we retrain TradingMoE with five random seeds and analyze the resulting daily return series; the full protocol and per-seed results are given in Appendix B.5. Across seeds, TradingMoE attains a cumulative return of $+49.52\% \pm 3.30\%$ and a Sharpe ratio of 5.18 ± 0.44 (mean $\pm$ std); even the worst seed ($+45.72\%$, Sharpe 4.678) remains more than $2.5\times$ the strongest baseline, LightGBM ($+18.19\%$, 1.723). Testing the seed-averaged daily returns against LightGBM, the Ledoit-Wolf robust Sharpe test rejects equal performance at two-sided $p = 0.018$, and a HAC-adjusted mean test at $p = 0.007$; per seed, four of five runs are significant at the 5% level and all five at the 10% level. Stationary-block-bootstrap 95% confidence intervals place the seed-averaged cumulative return in $[+28.5\%, +75.1\%]$ and the Sharpe ratio above 4.73, so even the lower confidence bounds exceed the strongest baseline’s point estimates; these bounds are stable across bootstrap block lengths of 10–20 trading days. All remaining experiments use the reference seed ($+49.08\%$), which lies within one standard deviation of the five-seed mean.

E.3 Transaction Cost Sensitivity

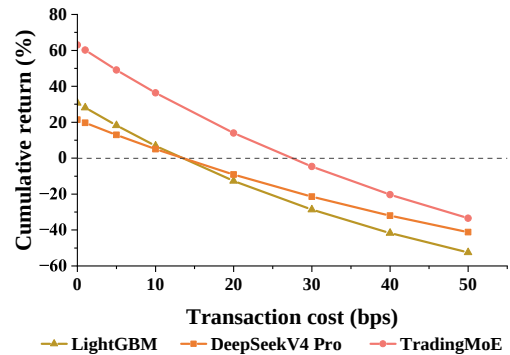


Figure 6: Cumulative return on Stock under varying one-way transaction-cost rates.

Since TradingMoE trades with an average daily turnover of 1.298, we examine how its advantage depends on the assumed cost level by re-running the Stock backtest under one-way transaction-cost rates from 0 to 50 bps, compared against the two strongest active baselines, LightGBM and DeepSeek V4 Pro. Figure 6 yields two

observations. First, TradingMoE degrades no faster than the baselines despite comparable turnover: its return remains positive up to roughly 28 bps, about twice the break-even cost of LightGBM and DeepSeek V4 Pro (both around 13–14 bps). Second, the advantage is not an artifact of the default cost setting: even when the cost rate is doubled from the default 5 bps to 10 bps, TradingMoE retains +36.34%, which still exceeds the best baseline evaluated at zero cost (+30.72%). At extreme rates beyond 30 bps, all active strategies fall below the passive NASDAQ benchmark, whose return is essentially cost-invariant; in this regime daily rebalancing itself, rather than any particular model, becomes unprofitable.